

Pitchly Project Guide

Codebase explanation, language overview, runtime flow, navigation map, and deployment notes.

Project root: C:\Users\ndong\Desktop\pitchly. This document describes the current repository after adding local run commands and deployment support.

Area	Technology	Purpose
Frontend	React, TypeScript, Vite	Browser app, routing, authentication screens, recording flow, dashboards, and results UI.
Backend	Python, FastAPI, Uvicorn, Gunicorn	API service, authentication verification, uploads, session orchestration, results, and health checks.
Storage/Auth	Supabase	User authentication, token validation, database connection, and private video storage.
Analysis	Gemini API placeholder plus Python pipeline	Session analysis workflow currently produces demo results through a background job.
Deployment	Render, Vercel/Netlify helpers	Production build and hosting configuration for frontend and backend.

1. High-Level Architecture

Pitchly is split into two applications under one repository. The `pitchly` folder is the client-side React application. The `pitchly-backend` folder is the FastAPI backend. The root folder now provides shared convenience scripts, documentation, and deployment configuration.

- The browser loads the Vite React app and uses React Router for page navigation.
- Supabase handles login, signup, password reset, email verification, and browser-side auth sessions.
- The frontend attaches the Supabase bearer token to backend API requests through an Axios interceptor.
- The backend verifies bearer tokens with Supabase admin credentials before allowing protected requests.
- Recorded videos are uploaded to Supabase Storage through backend upload endpoints.
- Session analysis runs as a FastAPI background task and stores in-memory demo results through `services/session\_store.py`.

Request Flow

```
User action in React -> Axios API client -> FastAPI router -> Supabase token verification ->
service layer -> result/status response -> React polling or results display.
```

Environment Flow

- Frontend variables use the `VITE\_` prefix because Vite exposes only prefixed variables to browser code.
- Backend variables are loaded by Pydantic Settings from `pitchly-backend/.env`.
- Production CORS is controlled by `CORS\_ORIGINS`, allowing a deployed frontend URL to call the API.

2. Frontend Code

The frontend is a TypeScript React single page app. It is built by Vite and served locally at `http://localhost:5173`.

File or Folder	Role
pitchly/src/main.tsx	Application bootstrap. It mounts React into <code>#root</code> and renders <code>App</code> inside <code>StrictMode</code> .
pitchly/src/App.tsx	Defines all client routes and wraps the app with <code>AuthProvider</code> .
pitchly/src/auth/*	Auth context, auth service, route protection, and hooks for consuming logged-in user state.
pitchly/src/pages/*	Full page screens: home, dashboard, mode selection, setup, session recording, processing, results, and auth pages.
pitchly/src/components/layout/*	Reusable page shell, sidebar, and top bar components.
pitchly/src/components/ui/*	Reusable visual controls such as buttons, inputs, cards, modal, badges, skeletons, and password strength.
pitchly/src/components/webcam/*	Camera preview and recording controls used during practice sessions.
pitchly/src/components/feedback/*	Feedback visualization components such as score cards, radar chart, and feedback panels.
pitchly/src/hooks/*	Reusable hooks for recording, session state, and polling backend status.
pitchly/src/services/api.ts	Central Axios API client. Adds Supabase bearer token to backend calls.
pitchly/src/lib/supabase.ts	Creates the Supabase browser client from Vite environment variables.
pitchly/src/lib/api.ts	Small helper to build backend API URLs.
pitchly/src/constants/index.ts	Shared app constants.
pitchly/src/index.css and App.css	Global and app-level styling.
pitchly/public/*	Static browser assets such as favicon, icons, and deployment redirects.

Frontend API Methods

- `analyzeSession` starts a backend session analysis job.
- `uploadVideo` sends a recorded video file for a specific session.
- `getStatus` checks background job progress.
- `getResults` fetches analysis output for a session.
- `getDashboard` loads dashboard data for the current user.

3. Navigation Map

Navigation is defined in ``pitchly/src/App.tsx``. Public routes are available to everyone. Protected routes require a valid Supabase session.

Route	Screen	Access	Purpose
/	Home	Public	Landing/start screen for the application.
/login	Login	Public	Sign in with Supabase credentials.
/signup	SignUp	Public	Create an account.
/forgot-password	ForgotPassword	Public	Request password reset email.
/reset-password	ResetPassword	Public	Set a new password after reset.
/verify-email	VerifyEmail	Public	Email verification flow.
/dashboard	Dashboard	Protected	Shows user practice activity and dashboard metrics.
/modes	ModeSelection	Protected	Choose the type of practice session.
/interview-setup	InterviewSetup	Protected	Configure interview practice.
/social-setup	SocialSetup	Protected	Configure social speaking practice.
/session	Session	Protected	Record or run the active practice session.
/processing	Processing	Protected	Wait for background analysis.
/results/:sessionId	Results	Protected	Show scores and feedback for a completed session.
*	Navigate to /	Fallback	Unknown routes redirect home.

Typical User Journey

Home -> signup/login -> dashboard -> modes -> setup -> session recording -> processing -> results.

4. Backend Code

The backend is a FastAPI app served locally with Uvicorn and deployable with Gunicorn plus the Uvicorn worker. Its local URL is `http://localhost:8000`.

File or Folder	Role
pitchly-backend/main.py	Creates the FastAPI app, configures CORS, includes routers, and exposes <code>/health</code> .
pitchly-backend/app/config.py	Typed environment settings loaded from <code>.env</code> , including Supabase, database, and CORS settings.
pitchly-backend/app/auth.py	Verifies Supabase bearer tokens for protected endpoints.
pitchly-backend/app/supabase_client.py	Creates a cached Supabase admin client using the service role key.
pitchly-backend/app/storage.py	Validates video uploads, writes to Supabase Storage, and creates signed URLs.
pitchly-backend/routers/auth.py	Receives protected auth event logs.
pitchly-backend/routers/session.py	Starts analysis jobs and exposes job status.
pitchly-backend/routers/upload.py	Receives session video upload files.
pitchly-backend/routers/results.py	Returns analysis results for a session.
pitchly-backend/routers/dashboard.py	Returns dashboard data for the current user.
pitchly-backend/services/session_store.py	In-memory session, job, dashboard, and demo result storage.
pitchly-backend/services/analysis_pipeline.py	Background analysis pipeline. Currently creates demo results after a short delay.
pitchly-backend/schemas/session.py	Pydantic request models for session creation.
pitchly-backend/migrations/*	Alembic database migration files for the Supabase/Postgres schema.
pitchly-backend/requirements.txt	Pinned Python dependencies for local and production installs.

Backend Health Check

```
GET /health -> returns status, environment, and active Supabase storage bucket.
```

5. How Core Features Work

Authentication

- The frontend uses Supabase browser authentication from ``pitchly/src/lib/supabase.ts``.
- Protected pages are wrapped in ``ProtectedRoute``, so unauthenticated users cannot access core app screens.
- Backend calls include ``Authorization: Bearer `` through the Axios interceptor.
- The backend validates the token in ``app/auth.py`` using the Supabase admin client.

Session Recording and Upload

- The session screen uses webcam and recorder components to capture video.
- The upload API sends a ``FormData`` payload to ``/upload/{sessionId}``.
- The backend checks content type and max file size before writing to Supabase Storage.
- Stored videos receive signed URLs for controlled access.

Analysis and Results

- The frontend calls ``/session/analyze`` to create a session and background job.
- FastAPI ``BackgroundTasks`` runs ``run_analysis_pipeline`` without blocking the HTTP response.
- The current pipeline writes demo results through ``create_demo_result``.
- The frontend polls ``/session/status/{jobId}`` and then navigates to ``/results/{sessionId}``.

Important Runtime Limitation

The current backend stores sessions and jobs in memory through ``session_store.py``. This works locally and for demos, but production persistence should move to Supabase/Postgres before relying on data surviving restarts or multiple backend instances.

6. Files Created or Changed for Local Run and Deployment

File	What it represents
README.md	Top-level project guide with local setup, build, and deployment instructions.
render.yaml	Render blueprint defining `pitchly-api` backend and `pitchly-web` static frontend services.
.gitignore	Ignores `.local/`, where local runtime logs can be written without dirtying git.
package.json	Root workspace scripts for frontend install, backend install, frontend dev, backend dev, and build.
package-lock.json	Updated root lock metadata to match the new root package name.
pitchly-backend/Procfile	Generic production process command for hosts that use Procfile conventions.
pitchly-backend/runtime.txt	Pins Python 3.11.9 for deployment platforms that read runtime files.
pitchly-backend/.env.example	Documents `CORS_ORIGINS` in addition to existing backend environment variables.
pitchly-backend/app/config.py	Adds `cors_origins` setting and `allowed_origins` parser.
pitchly-backend/main.py	Uses parsed `settings.allowed_origins` for FastAPI CORS.
pitchly/.nvmrc	Pins Node 20 for frontend deployments and developer machines using nvm.
pitchly/vercel.json	SPA rewrite so Vercel serves React routes through `index.html`.
pitchly/public/_redirects	SPA rewrite for Netlify-style static hosting.

Local Commands

```
npm run install:frontend npm run dev:backend npm run dev:frontend npm run build
```

7. Deployment Guide

Render Deployment

- Push the repository to GitHub.
- Create a Render Blueprint from the repository.
- Render reads `render.yaml` and creates both backend and frontend services.
- Set backend secrets on `pitchly-api` and frontend public Vite variables on `pitchly-web`.
- Set `VITE\_API\_URL` to the deployed backend URL.
- Set backend `CORS\_ORIGINS` to the deployed frontend URL.
- Verify `https://your-backend-url/health`, then open the frontend URL.

Required Backend Variables

```
GEMINI_API_KEY SUPABASE_URL SUPABASE_ANON_KEY SUPABASE_SERVICE_ROLE_KEY SUPABASE_STORAGE_BUCKET  
DATABASE_URL FRONTEND_URL CORS_ORIGINS ENVIRONMENT=production
```

Required Frontend Variables

```
VITE_SUPABASE_URL VITE_SUPABASE_ANON_KEY VITE_API_URL
```

Other Hosting Options

The backend can deploy to any Python web host that supports Gunicorn. The frontend can deploy to any static host that runs `npm ci && npm run build` and publishes `pitchly/dist`.

8. Maintenance Notes

- Keep `.env`` files out of git because they contain secrets.
- Use ``npm run build`` before deploying frontend changes.
- Use ``npm --prefix pitchly run lint`` before committing frontend changes.
- Replace in-memory session storage with database-backed persistence for production scale.
- Connect ``analysis_pipeline.py`` to the real Gemini/video/audio analysis logic when ready.
- Keep ``CORS_ORIGINS`` explicit in production rather than using a wildcard.
- Use Supabase storage bucket permissions carefully because uploaded practice videos may be sensitive.

Quick Local Verification

Backend: `http://localhost:8000/health` Frontend: `http://localhost:5173`